

线性回归

Linear Regression

孟庆鑫

人工智能学院



唐山学院
TANGSHAN UNIVERSITY

课件获取地址

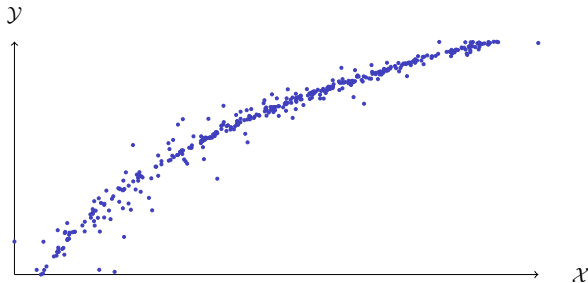


GitHub



Gitee

你真的会“预测”吗？



给你这样一组数据点，你会怎么做？

- 画一条直线？曲线？
- 还是凭感觉“差不多就行”？

问题是：你的“感觉”，能稳定的做对吗？

直觉并不稳定

同一组数据，不同的人可能会给出完全不同的拟合结果：

- 有人更偏向整体趋势
- 有人更关注局部点
- 有人试图“穿过所有点”

我们缺少一个统一的标准，来定义“什么是好的拟合”

问题的本质是什么？

我们真正面对的，不是“画一条线”，而是：

核心问题

给定有限数据，如何构造一个函数
使它不仅解释已有数据，还能预测未来？

形式化地，我们假设存在某个未知关系：

$$f: \mathcal{X} \rightarrow \mathcal{Y}, \quad \mathcal{X} \subset \mathbb{R}^m, \quad \mathcal{Y} \subset \mathbb{R}^n$$

但我们只能观测到：

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^N$$

我们要做的，是从“有限样本”逼近“未知规律”

如何表达“可能的规律”？

在不知道真实函数 f 的情况下，
我们必须先回答一个更基础的问题：

我们允许“什么样的函数”作为候选？

例如：

- 线性函数 $y \propto x$?
- 对数函数 $y \propto \ln x$?
- 平方根函数 $y \propto \sqrt{x}$?

学习问题 = 在一堆函数中寻找最优那个

线性回归的真正思想

如果我们不确定哪种函数是对的，一个自然策略是：

核心理念

把“所有可能的简单模式”组合起来，让数据自己决定权重

例如：

$$y = \theta_0 + \theta_1 x + \theta_2 \ln x + \theta_3 \sqrt{x}.$$

这意味着：

- 我们不再“选模型”
- 而是构造一个模型空间

线性回归，本质是学习函数的线性组合

线性回归模型的数学形式

更一般地，我们选取一组特征映射：

$$\phi_1(x), \phi_2(x), \dots, \phi_d(x),$$

构造模型：

$$h(x; \theta) = \theta_0 + \theta_1 \phi_1(x) + \dots + \theta_d \phi_d(x).$$

向量形式：

$$h(x; \theta) = \theta \phi(x).$$

“线性”不是关于 x ，而是关于参数 θ 。

回顾：Boosting

在上一讲中，我们构造了一个打分函数：

$$F(x) = \sum_{t=1}^T \alpha_t h_t(x),$$

其中：

- $h_t(x)$ ：第 t 个决策树
- α_t ：对应的权重

模型是“逐步构造”的：每一步引入一个新的函数

复杂性的来源

Boosting 的两个“困难来源”：

- 我们需要不断寻找新的函数 h_t
- 同时还要确定对应的权重 α_t

函数在变，权重也在变

固定函数

反过来

固定函数

预先固定一组函数 $\{\phi_1(x), \dots, \phi_d(x)\}$ ，只学习它们的组合方式

函数不再变化，只调整权重

Boosting vs 线性回归

Boosting	线性回归
逐步添加函数	预先固定函数
函数空间不断扩展	函数空间一次性给定
结构学习 + 参数学习	仅参数学习

Boosting 在“构造函数”，线性回归在“组合函数”

复杂性消失之后

一旦函数 $\phi(x)$ 固定：

- 模型结构已确定
- 唯一需要做的，是选择参数 θ

问题变为：

$$\min_{\theta} \frac{1}{2N} \sum_{i=1}^N \|y^{(i)} - \theta \phi(x^{(i)})\|_2^2$$

我们从“结构搜索问题”，退化为“参数优化问题”

这就是最小二乘。

最小二乘问题的矩阵形式

$$\Phi = [\phi(x^{(1)}), \phi(x^{(2)}), \dots, \phi(x^{(N)})]^\top = \begin{bmatrix} 1 & \phi_1(x^{(1)}) & \dots & \phi_d(x^{(1)}) \\ 1 & \phi_1(x^{(2)}) & \dots & \phi_d(x^{(2)}) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \phi_1(x^{(N)}) & \dots & \phi_d(x^{(N)}) \end{bmatrix}$$

$$Y = [y^{(1)}, y^{(2)}, \dots, y^{(N)}]^\top$$

此时单样本预测满足 $y^{(i)} = \theta \phi(x^{(i)}) + \epsilon^{(i)}$ ，所有样本的预测满足

$$Y^\top = \theta \Phi^\top + E^\top, \quad \text{其中} \quad E = [\epsilon^{(1)}, \epsilon^{(2)}, \dots, \epsilon^{(N)}]^\top \in \mathbb{R}^{N \times n}.$$

最小二乘的目标函数可以写成

$$J(\theta) = \frac{1}{2N} \sum_{i=1}^N \|\epsilon^{(i)}\|_2^2 = \frac{1}{2N} \sum_{i=1}^N (\epsilon^{(i)})^\top \epsilon^{(i)} = \frac{1}{2N} \text{tr}(EE^\top)$$

极值的本质

极值的本质

对于可微函数 $J(\theta)$ ，极小值点 $\hat{\theta}$ 满足：

沿任意方向 Δ 的一阶变化为零

考虑扰动 $\theta + t\Delta$ ，其中 t 是小标量， Δ 是任意同型矩阵

若 $\hat{\theta}$ 是最小值点，则对 $\forall \Delta$ 有 $\left. \frac{d}{dt} J(\theta + t\Delta) \right|_{t=0} = 0$ ，即

$$\lim_{t \rightarrow 0} \frac{J(\theta + t\Delta) - J(\theta)}{t} = 0$$

对 $J(\theta)$ 求一阶变分

$$\begin{aligned}J(\theta) &= \frac{1}{2N} \operatorname{tr}(EE^\top) \\J(\theta + t\Delta) &= \frac{1}{2N} \operatorname{tr}((E - t\Phi\Delta^\top)(E - t\Phi\Delta^\top)^\top) \\&= \frac{1}{2N} \operatorname{tr}(EE^\top - tE\Delta\Phi^\top - t\Phi\Delta^\top E^\top + t^2\Phi\Delta^\top\Delta\Phi^\top) \\&= \frac{1}{2N} \left(\operatorname{tr}(EE^\top) - 2t\operatorname{tr}(E\Delta\Phi^\top) + t^2\operatorname{tr}(\Phi\Delta^\top\Delta\Phi^\top) \right)\end{aligned}$$

得到

$$\lim_{t \rightarrow 0} \frac{J(\theta + t\Delta) - J(\theta)}{t} = -\frac{1}{2N} \operatorname{tr}(E\Delta\Phi^\top) = 0$$

最小二乘的解析解

求解 $\text{tr}(E\Delta\Phi^\top) = 0$, $\forall \Delta$, 得到 $E^\top\Phi = 0$

在 $\Phi^\top\Phi$ 可逆的条件下, 最小二乘问题具有解析解:

$$\hat{\theta} = Y^\top\Phi(\Phi^\top\Phi)^{-1}.$$

一个危险的信号：解真的可靠吗？

我们刚刚得到了最小二乘解：

$$\hat{\theta} = Y^T \Phi (\Phi^T \Phi)^{-1}$$

这个解看起来非常“干净”，但它隐含了两个强假设：

- $\Phi^T \Phi$ 可逆
- 数据足以稳定确定 θ

问题是：这在现实中不一定成立

解对数据过于敏感

当特征之间高度相关，或维度较高时：

$$\Phi^T \Phi \approx \text{singular}$$

会发生什么？

- $(\Phi^T \Phi)^{-1}$ 数值爆炸
- θ 变得极端（非常大）
- 模型对微小扰动极其敏感

这就是过拟合的代数本质

拟合得好，模型就好吗？

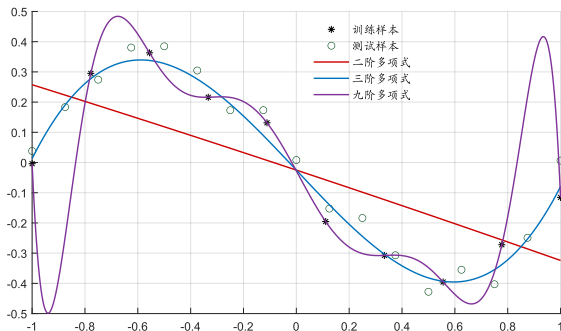
我们刚刚看到：

- 最小二乘可以让训练误差非常小
- 但同时可能导致参数 θ 极端不稳定

一个在训练集上完美的模型，可能在新数据上彻底失效

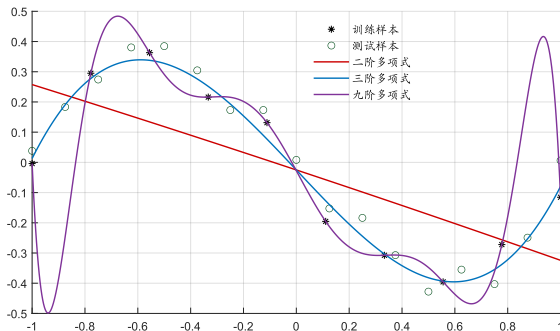
这正是泛化问题的核心

模型复杂度的影响：多项式拟合示例



- 在同一数据集上，采用不同阶数的多项式进行拟合
- 模型阶数越高，拟合训练数据的能力越强

过拟合与泛化能力



高阶模型在训练集上误差很小，但预测误差反而显著增大。这种现象被称为过拟合

仅仅“拟合训练数据”并不足以评价模型，我们真正关心的是泛化能力

泛化问题与正则化的动机

- 过拟合并不意味着模型形式“错误”
- 问题在于模型自由度过大，对噪声过于敏感
- 我们希望保留模型较强的表达能力
- 同时抑制模型对噪声的过度拟合

问题不在于模型太强，而在于解太“极端”

在最小二乘之外加入约束

岭回归 (Ridge Regression) 目标函数

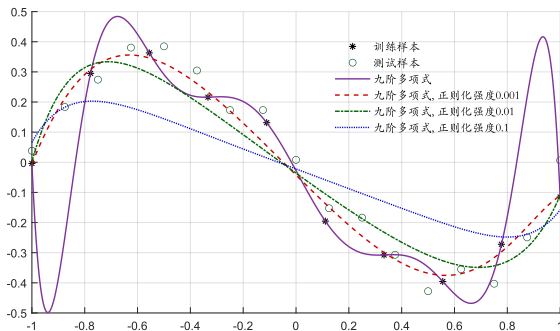
$$\min_{\theta} \frac{1}{2N} \sum_{i=1}^N \|y^{(i)} - \theta \phi(x^{(i)})\|_2^2 + \frac{\lambda}{2} \|\theta\|_F^2$$

- 第二项为参数范数惩罚，用于约束模型复杂度
- 正则化系数 λ 控制约束强度
- 我们不仅要求“拟合数据”，还要求“参数不要太大”

正则化不是限制模型，而是改变“我们相信什么样的解”

λ 在拟合能力与泛化能力之间提供连续调节机制

正则化参数的可视化效果



- 固定数据集，仅改变正则化参数 λ ，观察模型拟合曲线随 λ 的变化。
- 正则化使模型逐渐变得更加平滑和稳定。

正则化改变了什么？

最小二乘解：

$$\hat{\theta} = Y^T \Phi (\Phi^T \Phi)^{-1}$$

岭回归解：

$$\hat{\theta} = Y^T \Phi (\Phi^T \Phi + \lambda NI)^{-1}$$

我们在“逆矩阵”上加了一个缓冲项

这带来了：

- 数值稳定性提升
- 解不再极端
- 对噪声不敏感

为什么正则化是必要的？

- 最小二乘可以完美拟合数据
- 但解可能极不稳定，对噪声高度敏感

正则化通过“限制参数”，提升模型的稳定性
在真实应用中，稳健性往往比极致拟合更重要

但我们付出了什么代价？

加入正则化后，我们优化的是：

$$\min_{\theta} \frac{1}{2N} \sum_{i=1}^N \|y^{(i)} - \theta \phi(x^{(i)})\|^2 + \frac{\lambda}{2} \|\theta\|^2$$

我们不再完全拟合数据，而是“偏向”某些解

这意味着：估计产生了偏差（bias）

思考

如果我们已经知道：
正则化会引入偏差（bias）

那是否意味着
正则化不应该被使用？

答案取决于问题本身

在机器学习中：

- 我们更关心预测性能（泛化能力）
- 允许引入偏差来降低方差

在某些科学与工程问题中：

- 参数具有明确的物理意义
- 必须保证无偏估计

正则化不是“默认选项”，而是一种建模立场

回到岭回归

回顾岭回归：

$$\min_{\theta} J(\theta)$$

其中 $J(\theta)$ 是一个关于 θ 的二次函数

问题：如何在高维空间中找到 $J(\theta)$ 的最小点？

如果你只能“局部观察”，该怎么办？

假设我们站在当前点 θ_t ，我们只知道

- 当前函数值 $J(\theta_t)$
- 局部变化趋势 $\nabla J(\theta_t)$

我们该往哪个方向走？

如果你只能“局部观察”，该怎么办？

假设我们站在当前点 θ_t ，我们只知道

- 当前函数值 $J(\theta_t)$
- 局部变化趋势 $\nabla J(\theta_t)$

我们该往哪个方向走？

答案：沿着函数下降最快的方向

梯度下降的基本思想

最陡下降方向为 $-\nabla J(\theta_t)$ ，因此更新为：

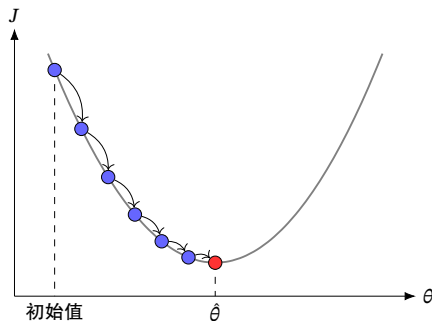
$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla J(\theta_t)$$

其中 $\eta > 0$ 为学习率。

- 负梯度方向对应局部最快下降方向
- 每一步沿该方向移动一小段距离

每一步都在用局部信息逼近最优

梯度下降的直观示意



参数沿着目标函数下降最快的方向逐步逼近最优解。

岭回归的梯度下降更新

对岭回归目标函数求梯度，可得参数更新规则：

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{N} (\theta_t \Phi^\top - Y^\top) \Phi - \eta \lambda \theta_t.$$

- 正则项引入了额外的收缩修正
- 有助于提升数值稳定性与泛化性能

二次函数 + 梯度下降 = 全局最优可达

计算开销与实践困境

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{N} (\theta_t \Phi^\top - Y^\top) \Phi - \eta \lambda \theta_t.$$

- 每次梯度计算涉及矩阵乘法
- 当样本数 N 很大时，计算成本较高
- 全量梯度在大规模数据下并不高效

实际系统中通常采用近似梯度方法。

实践中的替代方案

我们用一小部分数据近似整个数据
来计算梯度
这就是小批量梯度下降思想

小批量梯度下降 (Mini-batch GD)

- 将数据随机打乱并分成若干小批量
- 每次仅使用一个批量估计梯度

$$[\Phi \quad Y] \xrightarrow{\text{行随机重排}} [\Phi_j \quad Y_j] \xrightarrow{\text{每 } M \text{ 行划为一块}} \begin{bmatrix} \Phi_j^1 & Y_j^1 \\ \cdots & \cdots \\ \Phi_j^2 & Y_j^2 \\ \cdots & \cdots \\ \vdots & \vdots \end{bmatrix}.$$

对第 i 个批量，有更新规则：

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{M} \left(\theta_t (\Phi_j^i)^\top - (Y_j^i)^\top \right) \Phi_j^i - \eta \lambda \theta_t.$$

这是现代机器学习中最常用的优化策略之一。

随机梯度下降 (Stochastic GD)

- 当批大小 $M = 1$ 时，称为随机梯度下降
- 每次只抽取一个样本用于更新，成本极低
- 梯度估计存在噪声，但计算效率极高

SGD 的本质是用“随机性”换取“计算效率”

我们刚刚做了什么？

无论是：

- 解析解（最小二乘）
- 梯度下降（GD / SGD）

它们都是在优化参数 θ

本质：在参数空间中寻找最优 θ

我们真的需要 θ 吗?

我们的目标只是找一个预测函数

$$h(x) = \theta \phi(x)$$

我们真的需要显式求出 θ 吗?

还是说，我们可以绕过 θ ?

从岭回归的解重新出发

回顾岭回归解：

$$\hat{\theta} = Y^T \Phi (\Phi^T \Phi + \lambda NI)^{-1}$$

代入预测函数 $h(x) = \hat{\theta} \phi(x)$ ：

$$h(x) = Y^T \Phi (\Phi^T \Phi + \lambda NI)^{-1} \phi(x)$$

关键重排

利用矩阵恒等式

$$\Phi (\Phi^T \Phi + \lambda NI)^{-1} = (\Phi \Phi^T + \lambda NI)^{-1} \Phi$$

得到：

$$h(x) = Y^T (\Phi \Phi^T + \lambda NI)^{-1} \Phi \phi(x)$$

完全消除 θ

$$h(x) = Y^T(\Phi \Phi^T + \lambda NI)^{-1} \Phi \phi(x)$$

注意:

$$\Phi \Phi^T = \begin{bmatrix} \phi(x^{(1)})^T \\ \phi(x^{(2)})^T \\ \vdots \\ \phi(x^{(N)})^T \end{bmatrix} [\phi(x^{(1)}), \phi(x^{(2)}), \dots, \phi(x^{(N)})]$$
$$\Phi \phi(x) = \begin{bmatrix} \phi(x^{(1)})^T \\ \phi(x^{(2)})^T \\ \vdots \\ \phi(x^{(N)})^T \end{bmatrix} \phi(x)$$

预测完全由“内积”决定

核函数出现

定义：核函数

$$k(a, b) = \phi(a)^\top \phi(b)$$

并引入 Gram 矩阵 $K = \Phi \Phi^\top$ ，则预测函数为：

$$h(x) = Y^\top (K + \lambda NI)^{-1} \kappa(x)$$

其中：

$$\kappa(x) = \begin{bmatrix} k(x^{(1)}, x) \\ \vdots \\ k(x^{(N)}, x) \end{bmatrix}$$

我们完全绕过了 $\phi(x)$ 和 θ

我们在核空间中优化什么？

我们已经不再显式使用 θ ，并将模型写成：

$$h(x) = A \begin{bmatrix} k(x^{(1)}, x) \\ \vdots \\ k(x^{(N)}, x) \end{bmatrix}$$

其中 $A = Y^T(K + \lambda NI)^{-1}$

我们真正需要学习的是样本如何组合（矩阵 A ）

但我们不想通过“求逆矩阵”来得到它

从参数到样本表示

我们将参数写成：

$$\theta = Y^T \Phi (\Phi^T \Phi + \lambda NI)^{-1} = Y^T (K + \lambda NI)^{-1} \Phi = A \Phi$$

将其代入岭回归目标函数

$$\min_{\theta} \frac{1}{2N} \|Y^T - \theta \Phi^T\|_F^2 + \frac{\lambda}{2} \|\theta\|_F^2$$

可以得到：

$$\min_A \frac{1}{2N} \|Y^T - A K\|^2 + \frac{\lambda}{2} A K A^T$$

发生了什么变化？

原问题	核方法
优化 θ	优化 A
参数空间	样本空间
设计矩阵 Φ	Gram 矩阵 $K = \Phi \Phi^\top$

我们从“参数表示”切换到了“样本表示”

核岭回归的梯度下降

目标函数：

$$J(A) = \frac{1}{2N} \|Y^\top - AK\|_F^2 + \frac{\lambda}{2} AK A^\top$$

梯度为：

$$\nabla_A J(A) = -\frac{1}{N} (Y^\top - AK) K + \lambda AK$$

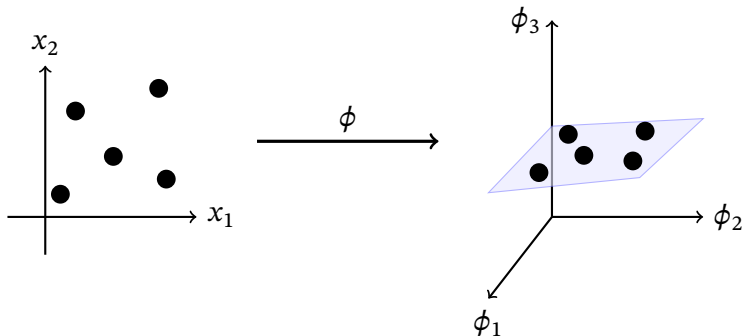
更新法则：

$$A_{t+1} \leftarrow A_t - \eta \nabla_A J(A_t)$$

优化方法没有改变，只是作用空间发生了变化

回看线性回归：从数据空间到特征空间

引入特征映射 ϕ ，将数据映射到特征空间



数据在原空间无明显结构，在特征空间中变为“近似线性”

核方法不再需要 ϕ

核方法的计算中，我们真正用到的是

$$k(x, x') = \phi(x)^\top \phi(x') = \langle \phi(x), \phi(x') \rangle$$

而不是 $\phi(x)$ 本身

核方法只需定义内积 $K(x, x')$

常用的核函数

- 多项式核

$$k(x, z) = (1 + x^T z)^d, \quad d \in \mathbb{N}$$

- 径向基函数核 (Radial Basis Function, RBF)

$$k(x, z) = \exp(-\gamma \|x - z\|^2)$$

多项式核

$$k(x, z) = (1 + x^\top z)^d, \quad d \in \mathbb{N}$$

考虑一维输入 $x, z \in \mathbb{R}$ ，利用二项式展开：

$$(1 + xz)^d = \sum_{k=0}^d \frac{d!}{k!(d-k)!} x^k z^k = 1 + dxz + \cdots + x^d z^d$$

这对应的特征映射可以取为 $\phi_k(x) = \sqrt{\frac{d!}{k!(d-k)!}} x^k$

多项式核可以理解为用有限阶多项式来逼近非线性关系

这与我们熟悉的 Taylor 展开在形式上非常相似

但系数是通过数据学习得到的

径向基函数核

$$k(x, z) = \exp(-\gamma \|x - z\|^2)$$

RBF 核对应的是无限维特征空间
其理论解释超出本课程的教学范围

核方法的本质

核函数不是把数据送入高维
而是让我们直接在高维空间中工作

它定义了一个内积空间
从而决定了数据的几何结构

我们关心的从来不是坐标，而是内积结构

线性回归的建模是否还有什么问题？

平方损失是从哪里来的？

我们一直在优化：

$$\frac{1}{2N} \sum_{i=1}^N \|y^{(i)} - \theta \phi(x^{(i)})\|^2$$

为什么是平方？为什么不是绝对值？或其他形式？

答案：它其实来自一个概率模型

从噪声模型到平方损失

假设数据 $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ 满足

$$y^{(i)} = \theta \phi(x^{(i)}) + \epsilon^{(i)}$$

其中

$$\epsilon^{(i)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2 I)$$

则：

$$\begin{aligned} & p(y^{(i)} | x^{(i)}; \theta) \\ &= \frac{1}{(2\pi)^{n/2} \sigma^n} \exp\left(-\frac{1}{2\sigma^2} (y^{(i)} - \theta \phi(x^{(i)}))^T (y^{(i)} - \theta \phi(x^{(i)}))\right) \end{aligned}$$

最大化对数似然 = 最小化平方误差

平方损失的本质是 在高斯噪声假设下的最大似然估计

我们真的只相信数据吗？

最大似然的假设是：

参数 θ 是一个固定但未知的值

但现实中：

- 数据有限
- 参数可能不稳定

我们可以对参数本身也建模

从 MLE 到 MAP

我们引入参数先验 $p(\theta) = (2\pi\tau^2)^{-n(d+1)/2} \exp\left(-\frac{1}{2\tau^2}\|\theta\|_F^2\right)$

目标变为：

$$\begin{aligned} p(\theta | \mathcal{D}) &= \frac{p(\theta)p(\mathcal{D} | \theta)}{p(\mathcal{D})} = \frac{p(\theta) \prod_{i=1}^N p(x^{(i)}, y^{(i)} | \theta)}{p(\mathcal{D})} \\ &= \frac{p(\theta) \prod_{i=1}^N p(x^{(i)})p(y^{(i)} | x^{(i)}; \theta)}{p(\mathcal{D})} \propto p(\theta) \prod_{i=1}^N p(y^{(i)} | x^{(i)}; \theta) \\ &\propto \exp\left(-\frac{1}{2\sigma^2} \|Y^\top - \theta \Phi^\top\|_F^2 - \frac{1}{2\tau^2} \|\theta\|_F^2\right) \end{aligned}$$

这是最大后验估计 (Maximum A Posteriori, MAP)

重新理解正则化

- L2 正则 = Gauss 先验
- 岭回归 = Gauss 先验下的最大后验估计

正则化是我们对参数的“先验信念”

加入正则化意味着

我们不相信极端的参数值

不同的先验，不同的信念

如果：

$$p(\theta) \propto \exp(-\beta \|\theta\|_1)$$

则 MAP 等价于

$$\min_{\theta} \frac{1}{2\sigma^2} \|Y^T - \theta \Phi^T\|_F^2 + \beta \|\theta\|_1$$

- L1 正则 = Laplace 先验
- LASSO = Laplace 先验下的最大后验估计

统一视角

目标函数	概率假设	结果
平方误差	Gauss 噪声	最小二乘
平方误差 + L2 正则	Gauss 噪声 + Gauss 先验	岭回归
平方误差 + L1 正则	Gauss 噪声 + Laplace 先验	LASSO

优化问题是概率模型的另一种表达

一个被过度简化的例子

在教学中，我们常用线性回归来预测房价

- 输入：面积、地段、楼龄
- 输出：价格

但真实世界，远没有这么简单

预测本身会改变结果

假设模型预测判断

房价将要下跌

可能发生：

- 买家观望
- 卖家降价

这是索罗斯的反身性理论：

参与者的认知会影响事实，事实反过来影响认知

根本性的不同

预测天气 $\neq$ 预测房价

- 天气不会因为你的预测而改变
- 房价可能会
- 有些系统“被预测”，不会反应
- 有些系统“被预测”，会反应

模型是否“参与系统”，是本质区别

现实的复杂性

在很多系统中：

预测 → 决策 → 环境改变

例如：

- 推荐系统：推荐改变用户偏好
- 金融市场：预测影响交易行为
- 自动驾驶：预测改变交通动态

数据不再是“静态样本”，而是“动态生成”

经典假设正在崩塌

像线性回归这种监督学习通常假设：

- 数据独立同分布 (i.i.d.)
- 系统是静态的

但在金融 / 推荐等场景中：

- 预测本身就会影响数据
- 分布随时间变化

模型正在破坏自己的假设

机器学习的边界问题

预测是否会影响被预测的系统？

- 如果会，问题就不再是“预测”
- 而是“干预”

线性回归（以及大多数监督学习）隐含了一个前提：

模型只是观察者，不是参与者

我们不仅在用模型理解世界

模型也在改变世界

这才是机器学习真正的边界